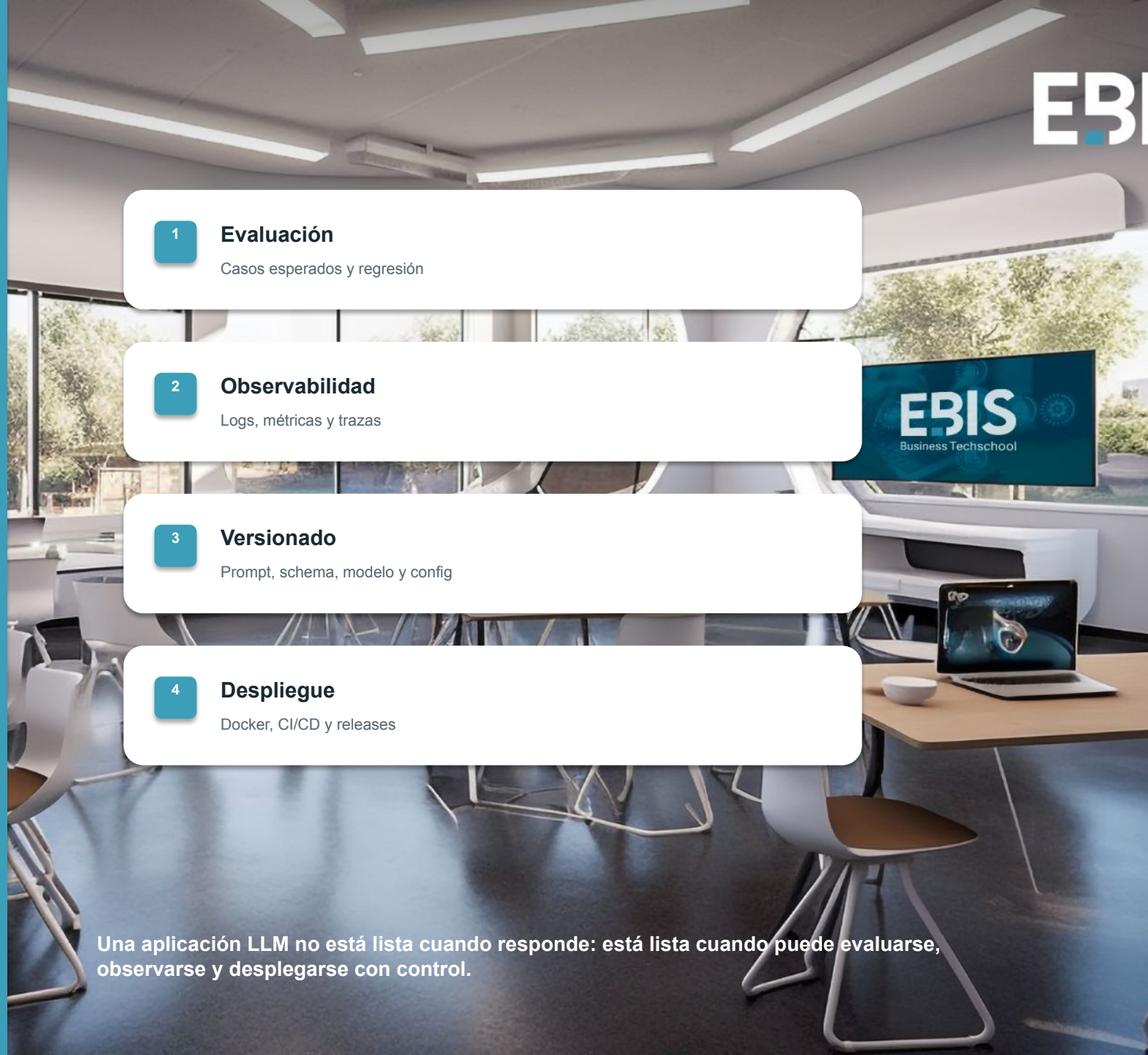
EBIS Business Techschool

Tema 4 · Clase 5

LLMOps, observabilidad y despliegue controlado

De la API funcional al servicio evaluable, medible y desplegable



Máster en Ingeniería de Automatización con IA Agéntica

www.ebiseducation.com

Clase 5

Cerramos el Tema 4 convirtiendo la API en un servicio operable.

EBIS Business Techschool

APIs LLM Python | Prompting + JSON | Contexto + sesiones | FastAPI + endpoints | **LLMOps + despliegue**

✓ Resultado de hoy

Una API LLM con evaluación básica, trazas útiles, versionado operativo, tests y Dockerfile.

! Importante

No vamos a hacer DevOps genérico. Vamos a operar el workflow LLM construido en Clase 4.

Cierre del tema

Pasamos de “funciona en local” a “puedo demostrar que sigue funcionando cuando cambian prompts, modelos, entradas, costes o latencias”.

Tesis de la clase: una app LLM no está lista cuando responde; está lista cuando puede evaluarse, observarse, versionarse y desplegarse con control.

Objetivos de aprendizaje de la sesión

Al terminar, debemos poder diseñar una estrategia mínima de LLMOps para nuestra API.

01 Entender LLMOps

Qué cambia frente a MLOps clásico y qué piezas hay que operar.

02 Diseñar observabilidad

Registrar latencia, tokens, errores, operación, versión y estado.

03 Evaluar workflows

Probar que el sistema produce estados y operaciones esperadas.

04 Preparar despliegue

Docker, tests, CI/CD y estrategias de release controladas.

Entregable práctico: observability.py · eval_cases.yml · run_eval.py · tests · Dockerfile · README de operación

El cambio clave: de servicio funcional a servicio operable

La API de Clase 4 ya responde; ahora necesitamos gobernar cambios, fallos y coste.

Clase 1
Invocar modelo

Clase 2
Controlar salida

Clase 3
Controlar estado

Clase 4
Exponer API

Clase 5
Operar servicio

Riesgo nuevo

Un cambio de prompt, modelo o contexto puede modificar decisiones sin romper el código.

Necesidad nueva

Medir comportamiento, comparar versiones y detectar degradaciones.

Práctica nueva

Tests, evaluación offline, trazas, métricas y Docker para despliegue.

El objetivo no es añadir herramientas por añadir: es poder responder “qué versión tomó esta decisión, con qué coste, con qué latencia y con qué resultado esperado”.

BLOQUE 1

LLMOps aplicado

Qué significa operar una aplicación LLM y por qué no basta con tener una API.

De demo funcional a sistema operable

Lo que funciona en una prueba puede fallar cuando cambian entradas, modelo, coste o latencia.

Demo local	Servicio operable
Probamos 3 mensajes	Evaluamos casos críticos y regresiones
Vemos respuesta manualmente	Medimos estado, operación, errores y coste
Una versión de prompt	Versionamos prompt, schema, modelo y config
Sin trazabilidad formal	Trace por request y por caso
Ejecuta en local	Docker, CI/CD y estrategia de release

Mensaje de diseño

Una app LLM falla de formas distintas a una API tradicional: puede responder válido en HTTP, pero incorrecto en negocio.

Respuesta operativa

Necesitamos gates de evaluación y trazas que detecten degradación semántica, no solo caídas técnicas.

LLMOps no es MLOps con otro nombre

Hay solape, pero las piezas que cambian son distintas.

Dimensión	MLOps clásico	LLMOps en nuestra API
Activo principal	Modelo entrenado y dataset	Prompt, schema, modelo externo, contexto y reglas
Evaluación	Métricas offline del modelo	Estados esperados, operación, seguridad y regresión
Drift	Datos/features cambian	Cambian inputs, modelo proveedor, prompt o contexto
Coste	Infra de entrenamiento/inferencia	Tokens, latencia, modelo y retries
Release	Modelo o pipeline	Prompt/model/config/schema/API contract

En LLMOps versionamos y evaluamos más que código: también prompts, schemas, herramientas, modelos, reglas de negocio y presupuestos de contexto.

Qué hay que operar en nuestra API LLM

Partimos de la API de Clase 4 y añadimos gobierno alrededor de cada pieza.

FastAPI endpoint
contrato HTTP

Session store
estado y trace

Prompt builder
contexto enviado

LLM client
modelo y latencia

Schema
salida estructurada

Business rules
routing seguro

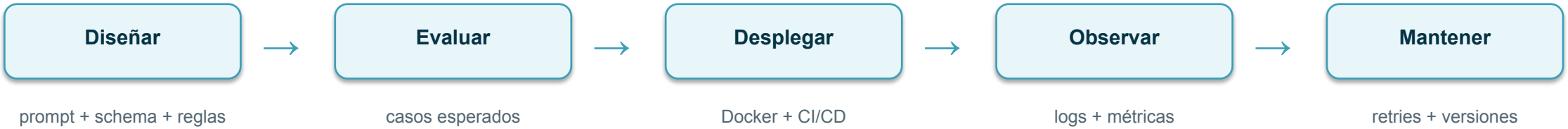
Evaluation set
casos esperados

Docker/CI
release reproducible

La pregunta operativa es: ¿puedo saber qué ha pasado, repetir la prueba, comparar versiones y desplegar sin perder control?

Ciclo de vida mínimo de una aplicación LLM

Diseñar, evaluar, desplegar, observar y mantener forman un ciclo continuo.



No lineal

La monitorización alimenta nuevas evaluaciones. Los errores reales deben convertirse en casos de regresión.

Criterio de madurez

No basta con “se ha desplegado”. Debemos poder pausar, volver atrás, comparar versiones y explicar decisiones.

Riesgos que LLMOps debe controlar

No todos los fallos son errores 500: algunos son decisiones incorrectas con respuesta válida.

Riesgo	Síntoma	Control mínimo
Calidad	operation incorrecta	eval set + regression gates
Coste	tokens crecen por contexto	context budget + coste por caso
Latencia	respuesta lenta	timeout + métricas p95
Seguridad	input intenta modificar reglas	validación + prompt boundaries
Estado	pierde campos entre llamadas	merge controlado + trace
Proveedor	rate limit / caída	retry, fallback o safe stop

La regla operativa: un fallo del modelo no debe avanzar el workflow ni corromper el estado del caso.

BLOQUE 2

Observabilidad

Convertir las trazas de la API en señales útiles para operar el servicio.

De trace local a observabilidad operativa

Clase 4 ya guardaba eventos. Ahora decidimos qué señales deben ser explotables.

trace

```
trace_event = {
  "case_id": case_id,
  "operation": operation,
  "state": state.model_dump(),
  "latency_ms": result.latency_ms,
  "input_tokens": result.input_tokens,
  "output_tokens": result.output_tokens,
  "model": result.model
}
```

Log

Registro de algo que ha ocurrido: request, error, operación o decisión.

Metric

Número agregable: latencia media, error rate, token rate, human review rate.

Trace

Secuencia de pasos de una request/caso para depurar el flujo completo.

Observabilidad no es guardar todo. Es guardar lo suficiente para depurar, medir y tomar decisiones operativas.

Qué registrar por request LLM

Cada llamada debe dejar una huella mínima: qué versión, qué operación y qué coste tuvo.

Campo	Para qué sirve
case_id	Agrupar eventos de un mismo workflow
event_type / operation	Entender qué hizo el sistema
status / error_type	Separar éxito, timeout, rate limit o salida inválida
model / prompt_version	Depurar cambios de comportamiento
latency_ms / tokens	Controlar UX, coste y capacidad
schema_validation_ok	Saber si el output era consumible
requires_human_review	Medir automatización segura vs escalado

Si no queda en log o trace, en producción no ha ocurrido. Y si no tiene versión, no se puede comparar.

Métricas por capa: no todo mide lo mismo

El dashboard debe separar salud técnica, coste y calidad del workflow.

Técnica

status_code
timeout_rate
provider_error_rate
p95 latency

Coste

input_tokens
output_tokens
cost_per_case
retry_rate

Workflow

operation_distribution
human_review_rate
ready_rate
missing_fields

Calidad

expected_operation_match
invalid_output_rate
invented_id_rate
regression_failures

Una API puede estar operativa y aun así estar tomando malas decisiones. Por eso necesitamos métricas técnicas y de negocio.

Dashboards y alertas: qué mirar primero

No hace falta empezar con una plataforma compleja: empieza con señales accionables.

Señal	Alerta razonable	Posible acción
invalid_output_rate	sube respecto a baseline	rollback prompt/model o activar repair
p95 latency	supera umbral	revisar contexto, proveedor o modelo
human_review_rate	sube abruptamente	analizar nuevos inputs o reglas
cost_per_case	sube por tokens	resumir, recortar contexto, enrutar modelos
provider_error_rate	429/503 frecuentes	backoff, fallback, cuotas

Principio

Una alerta debe tener una acción asociada. Si nadie sabe qué hacer, es ruido.

Conexión Clase 5

Los outputs de evaluación se convierten en baseline; producción se compara contra baseline.

BLOQUE 3

Evaluación

Probar workflows LLM por estado y operación, no solo por “respuesta bonita”.

Probar manualmente no es evaluar

Tres llamadas que funcionan en Swagger no garantizan que el sistema resista cambios.

Prueba manual

Útil para entender el flujo, pero no detecta regresiones cuando cambia el prompt o el modelo.

Evaluación offline

Ejecuta casos esperados y compara process_type, operation, missing_fields y seguridad.

Regression gate

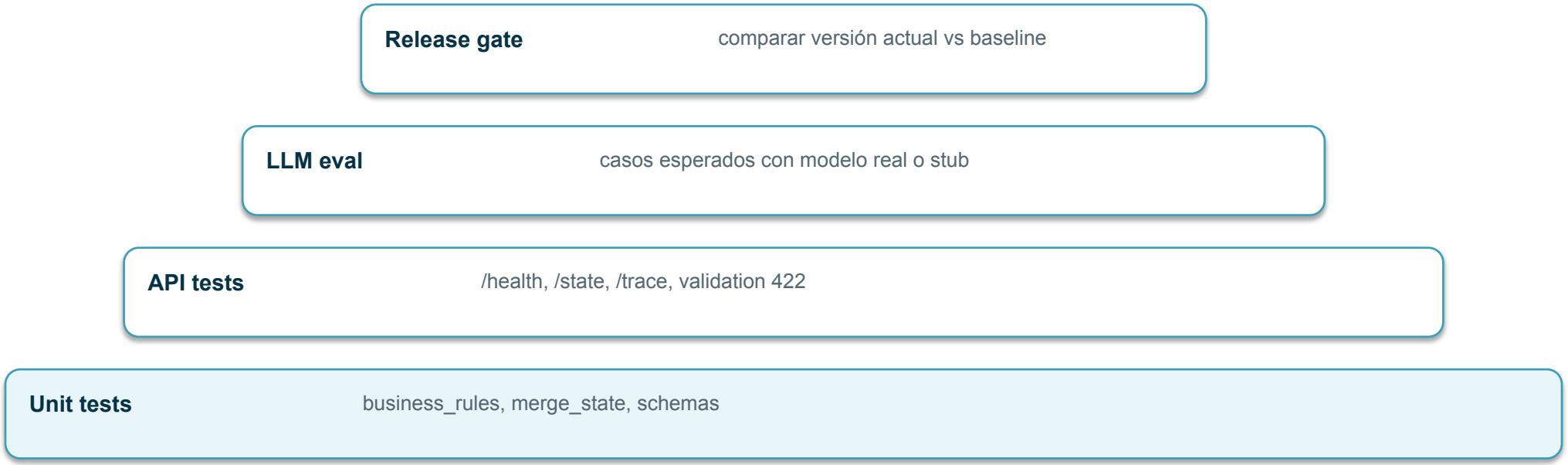
Si un cambio empeora casos críticos, no se despliega aunque la API arranque.

En LLM workflows, la pregunta no es “¿responde algo razonable?” sino “¿termina en el estado y operación correctos?”.

Pirámide de evaluación para la API LLM

Combinamos tests deterministas con evaluación de comportamiento del modelo.

Más abajo: tests rápidos y deterministas. Más arriba: pruebas más caras, pero más cercanas al comportamiento real.



Golden dataset: casos mínimos que no deben romperse

Un dataset pequeño y bien elegido vale más que cien ejemplos redundantes.

eval_cases.yml

```yaml
- case_id: EVAL-001
  input: "I want to return an item"
  expected:
    process_type: refund_request
    operation: ask_for_order_id
    missing_fields: [order_id, reason]

- case_id: EVAL-002
  input: "I was charged twice for order 9901"
  expected:
    process_type: billing_issue
    operation: send_to_human_review
```

Cubre intención

refund, delivery, billing, support, ambiguous.

Cubre riesgo

alta prioridad, datos ausentes, prompt injection, IDs inventados.

Cubre routing

ask_missing, execute, human_review, general_queue.

El dataset de evaluación es un contrato de comportamiento: define lo que la app no puede dejar de hacer.

Métricas de evaluación: medir el workflow, no solo el texto

El assistant_message puede variar; el estado y la operación deben ser estables.

Métrica	Qué comprueba
process_type_match	La intención se clasificó correctamente
operation_match	El routing final es el esperado
missing_fields_match	El sistema pide lo que falta
no_invented_id	No crea order_id si no aparece
human_review_match	Escala casos de riesgo
schema_valid	El output se puede consumir por software

La evaluación debe tolerar variación lingüística, pero no variación arbitraria en campos que gobiernan el proceso.

Release gates: cuándo bloquear un cambio

Una versión nueva de prompt/modelo debe superar mínimos antes de llegar a producción.

Gate	Umbral ejemplo	Decisión
schema_valid_rate	100% en casos críticos	Bloquear si falla
operation_match	≥ 95% global	Revisar si baja
invented_id_rate	0%	Bloquear
p95 latency	< umbral UX	Optimizar modelo/contexto
cost_per_case	≤ presupuesto	Reducir tokens o routing

Principio de release

No desplegamos una versión porque “parece mejor”; la desplegamos porque supera criterios explícitos de calidad, seguridad, latencia y coste.

BLOQUE 4

Versionado, errores y coste

Controlar qué cambió, cómo falla y cuánto cuesta operar el servicio.

Versionar lo que afecta a la decisión

En LLMOps, una regresión puede venir de código, prompt, schema, modelo o configuración.

trace

```
trace_event = {
  "model": "claude-haiku-4-5-20251001",
  "prompt_version": "triage_prompt_v1",
  "schema_version": "conversation_update_v1",
  "rules_version": "business_rules_v1",
  "config_version": "class5_config_v1",
  "operation": operation
}
```

Prompt version

Cambia instrucciones y ejemplos.

Schema version

Cambia campos, tipos y contrato.

Model/config version

Cambia comportamiento, coste y latencia.

Sin versionado, una traza explica “qué ha pasado”, pero no “con qué configuración ha pasado”.

Errores y respuestas seguras

El objetivo no es evitar todo fallo, sino impedir que un fallo avance el workflow mal.

Error	Respuesta segura
Input inválido	422 · no llamar al LLM
Rate limit	backoff / retry controlado
Timeout	504 · retry o fallback
Output inválido	no actualizar estado; retry repair o safe stop
Alta incertidumbre	human_review
Proveedor caído	fallback model o degradación funcional

Regla: no se guarda estado nuevo hasta tener una salida validada y reglas aplicadas.

Retries, repair y fallback: no todo error se trata igual

Cada mecanismo tiene coste y riesgo; no se debe reintentar sin control.

Retry

Mismo prompt, mismo modelo. Útil para timeouts o errores transitorios.

Repair

Pedir corregir formato/salida. Útil si el JSON/schema falla.

Fallback

Cambiar modelo/proveedor. Útil ante caída o saturación.

Safe stop

No continuar y pedir revisión. Útil ante riesgo de negocio.

Lo que no hacemos: repetir indefinidamente, ocultar errores o actualizar estado con una salida no validada.

Control de coste: tokens, modelo y contexto

Optimizar no es solo recortar: es enviar el contexto correcto al modelo correcto.

cost

```
estimated_cost = (
    input_tokens  * input_price_per_1k  +
    output_tokens * output_price_per_1k
) / 1000

case_cost += estimated_cost
```

Context budget

Últimos N mensajes + resumen + estado. Evitar historial completo.

Model routing

Usar modelo barato para triage y potente para casos complejos.

Caching / reuse

Reutilizar summaries, estados y resultados deterministas.

Human review

Escalar evita automatizar mal y gastar tokens en bucles.

Coste operativo = tokens × modelo × retries × volumen. LLMOps decide dónde gastar capacidad.

Estrategias de release: canary, blue-green y autoscaling

No todas las versiones deben llegar al 100% de tráfico de golpe.

Estrategia	Idea	Aplicación LLM
Blue-green	dos entornos y cambio de tráfico	desplegar prompt/model/API v2 con rollback rápido
Canary	pequeño porcentaje usa versión nueva	5% con prompt_v2; comparar calidad/coste/error
Autoscaling	más réplicas según carga	escala API por tráfico; ojo con rate limits del proveedor

Métrica de canary

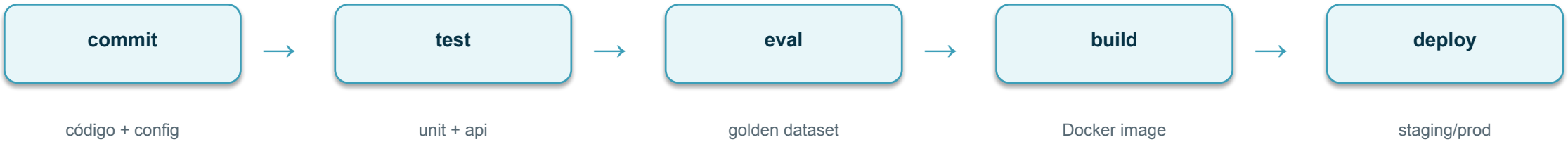
operation_match, invalid_output_rate, p95 latency, cost_per_case, human_review_rate.

Criterio

Si v2 empeora señales críticas, rollback aunque la API esté técnicamente sana.

CI/CD y Docker: reproducibilidad antes que complejidad

El objetivo de esta clase no es Kubernetes; es empaquetar y comprobar el servicio.



Un pipeline LLM debe validar más que “la API arranca”: debe ejecutar casos esperados y bloquear regresiones de comportamiento.

BLOQUE 5

Demo y práctica guiada

Añadimos evaluación, observabilidad y empaquetado a la API de Clase 4.

Caso práctico: AI Request Triage API encapsulado

No cambiamos de aplicación: reforzamos la API de Clase 4 para operarla.

Punto de partida

API FastAPI con /cases/{case_id}/events, state, trace y business rules.

Añadimos

versionado, observability.py, eval_cases.yml, run_eval.py, tests y Dockerfile.

Resultado

Un servicio evaluable, medible y empaquetable para despliegue controlado.

La práctica estará más guiada: cada snippet indica objetivo, TODO, hint y check de validación.

Estructura recomendada del repositorio

Partimos del servicio de Clase 4 y añadimos capa de operación.

project

```
class_05_llmops_service/
├── .env.example
├── config.yml
├── app.py
├── observability.py
├── eval_cases.yml
├── run_eval.py
├── tests/
│   ├── test_business_rules.py
│   └── test_api.py
├── Dockerfile
├── .dockerignore
└── README.md
```

Nuevo

observability.py, eval_cases.yml, run_eval.py, tests y Dockerfile.

Reutilizado

app.py, schemas.py, business_rules.py, llm_client.py y session_store.py.

Check

La API debe seguir ejecutando /health y /cases/{case_id}/events.

Paso 0 · requirements y .env: preparar operación

Añadimos dependencias para tests y mantenemos secretos fuera del código.

setup

```
# requirements.txt
fastapi
uvicorn[standard]
anthropic
python-dotenv
pydantic
pyyaml
pytest
httpx
pandas

# .env.example
LLM_PROVIDER=anthropic
LLM_API_KEY=replace_with_your_key
LLM_MODEL=claude-haiku-4-5-20251001
LLM_TIMEOUT_SECONDS=30


# For tests, evaluation and demos without provider cost:
USE_MOCK_LLM=false
```

Objetivo

Instalar lo necesario para API, tests y evaluación sin mezclar secretos con código.

TODO

Añadir pytest y httpx. Copiar .env.example a .env y rellenar API key local.

Hint

Los tests de API pueden usar TestClient/httpx sin llamar al LLM si aislamos dependencias.

Check

python -m pytest debe descubrir tests aunque al principio fallen por TODOs.

Paso 1 · config.yml: versiones y política operativa

La configuración debe decir no solo cómo llamar al modelo, sino qué versión estamos probando.

config.yml

```yaml
app:
  name: "AI Request Triage API"
  version: "1.1.0"
  deployment_environment: "local“


llm:
  temperature: 0.0
  max_tokens: 700


context:
  max_recent_messages: 6
  summarize_after_messages: 8
  keep_last_messages_after_summary: 4

versions:
  prompt_version: TODO
  schema_version: TODO
  rules_version: TODO
  eval_set_version: TODO


observability:
  request_log_path: "logs/requests.jsonl"
  include_prompt_preview: false
  prompt_preview_chars: 400

t:
```

Objetivo

Que cada traza se pueda asociar a una versión concreta de prompt, schema y reglas.

TODO

Completar versions.

Hint

Usa nombres explícitos: triage_prompt_v1, conversation_update_v1, rules_v1.

Check

El trace debe incluir estas versiones en cada evento procesado.

Paso 2 · observability.py: crear eventos de traza consistentes

Centralizamos el formato de observabilidad para no construir diccionarios distintos en cada endpoint.

observability.py

```python
def build_observability_event(*, case_id: str, event_type: str, message: str, operation:
str, state: dict, model: str, latency_ms: int, input_tokens: int, output_tokens: int,
status: str, config: dict, error_type: str | None = None, prompt: str | None = None) ->
dict[str, Any]:
    event = {
        "timestamp": datetime.now(timezone.utc).isoformat(),
        "deployment_environment": config["app"]["deployment_environment"],
        "case_id": case_id,
        "event_type": event_type,
        "message_chars": len(message),
        "operation": operation,
        "status": status,
        "error_type": error_type,
        "model": model,
        "latency_ms": latency_ms,
        "input_tokens": input_tokens,
        "output_tokens": output_tokens,
        "estimated_cost": estimate_cost(input_tokens, output_tokens, config),
        "currency": config["cost"]["currency"],
        "prompt_version": TODO,
        "schema_version": TODO,
        "rules_version": TODO,
        "state": state,
    }
    if config["observability"].get("include_prompt_preview") and prompt:
        chars = config["observability"].get("prompt_preview_chars", 400)
        event["prompt_preview"] = prompt[:chars]
    return event
```

Objetivo

Un único builder para logs/traces operativos.

TODO

Leer versiones desde config y añadir status/error_type si aplica.

Hint

No metas secretos ni prompts completos en trazas productivas.

Check

GET /trace debe mostrar versiones, tokens, latencia y operación.

Paso 3 · app.py: enriquecer trace sin cambiar el contrato principal

El endpoint sigue igual para el cliente, pero ahora deja señales operativas útiles.

app.py

```
obs_event = build_observability_event(
            case_id=case_id,
            event_type=request.event_type,
            message=request.message,
            operation=operation,
            state=state.model_dump(mode="json"),
            model=result.model,
            latency_ms=result.latency_ms,
            input_tokens=result.input_tokens,
            output_tokens=result.output_tokens,
            status="success",
            config=config,
            prompt=prompt,
        )
session.add_trace_event(obs_event)
write_jsonl(config["observability"]["request_log_path"], obs_event)
```

Objetivo

Mantener response simple, pero registrar suficiente información para operar.

TODO

Sustituir diccionario manual por build_observability_event().

Hint

El estado se actualiza solo después de validar el output y aplicar reglas.

Check

/cases/S-001/trace debe contener trace_event homogéneo.

Paso 4 · eval_cases.yml: definir casos esperados

El dataset de evaluación debe ser pequeño, claro y cubrir riesgos reales.

eval_cases.yml

```yaml
case_id: "EVAL-001"
    description: "Refund request completed after order and reason"
    events:
      - "I want to return an item."
      - "The order number is 8921."
      - "It arrived broken."
    expected:
      process_type: "refund_request"
      order_id: "8921"
      reason: "damaged_item"
      operation: "start_refund_validation"
      requires_human_review: false
      ready_for_next_step: true
```

Objetivo

Definir comportamiento mínimo aceptable de la API.

TODO

Añadir 6–10 casos: refund, delivery, billing, technical y prompt injection.

Hint

Evalúa campos de negocio; no obligues a que assistant_message sea idéntico.

Check

Cada caso debe tener expected.operation y al menos un campo esperado de state.

Paso 5 · run_eval.py: ejecutar evaluación offline

La evaluación compara el resultado del workflow contra expected fields.

run_eval.py

```python
def compare_expected(
    final_state: dict,
    operation: str,
    expected: dict
) -> tuple[bool, list[str]]:
    """
    Compara el estado final obtenido con el estado esperado.


    Devuelve:
        - True si todo coincide.
        - False y una lista de errores si encuentra diferencias.
    """


    errors = []


    # Recorremos cada dato que esperamos encontrar.
    for key, expected_value in expected.items():


        # El campo "operation" no está en final_state,
        # por eso usamos el parámetro operation.
        if key == "operation":
            actual_value = operation
        else:
            # Para el resto de campos buscamos su valor en final_state.
            actual_value = TODO


        # Si el valor obtenido no coincide con el esperado,
        # guardamos una descripción del error.
        if actual_value != expected_value:
            error = (
                f"{key}: "
                f"expected={expected_value!r}, "
                f"actual={actual_value!r}"
            )
            errors.append(error)
```

Objetivo

Producir un reporte simple: qué casos pasan y cuáles fallan.

TODO

Completar comparaciones contra expected y calcular métricas globales.

Hint

Al principio puede ejecutarse con una función local; luego contra la API real.

Check

outputs/eval_report.csv debe mostrar passed=True/False por caso.

Paso 6 · tests/test_business_rules.py: asegurar lógica determinista

Las reglas de negocio son código; deben probarse sin llamar al LLM.

pytest

```
def test_refund_request_missing_order_id():
    state = ConversationState(
        process_type=ProcessType.refund_request,
        reason="damaged_item",
        priority=Priority.medium
        )
    state = apply_business_rules(state)
    assert state.ready_for_next_step is False
    assert state.missing_fields == TODO
    assert determine_next_operation(state) == TODO


def test_high_priority_goes_to_human_review():
    # TODO: build high priority state
    # TODO: assert requires_human_review is True    state = apply_business_rules(state)
```

Objetivo

Garantizar que routing y readiness no dependen de la interpretación del modelo.

TODO

Cubrir missing_fields, high priority y operación esperada.

Hint

Estos tests deben ser rápidos y deterministas.

Check

pytest tests/test_business_rules.py debe pasar sin API key.

Paso 7 · tests/test_api.py: probar contrato HTTP

No todo test debe llamar al proveedor LLM; primero valida endpoints y errores.

pytest

```
from fastapi.testclient import TestClient
from app import app

client = TestClient(app)

def test_health():
    response = client.get("/health")
    assert response.status_code == 200
    assert response.json()["status"] == "ok"

def test_empty_message_returns_422():
    response = client.post("/cases/T-002/events", json={"message": ""})
    assert response.status_code == TODO
```

Objetivo

Comprobar contrato de API, validación de entrada y endpoints sin coste LLM.

TODO

Completar status esperado y añadir GET /state para caso inexistente.

Hint

Un mensaje vacío devuelve 422 porque Pydantic valida min_length.

Check

pytest debe detectar errores de contrato antes de probar LLM real.

Paso 8 · Dockerfile: empaquetar el servicio

Docker hace reproducible el entorno de ejecución, pero los secretos siguen fuera de la imagen.

Dockerfile

```
FROM python:3.11-slim

WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 8000
CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "8000"]
```

Objetivo

Ejecutar la API de forma reproducible en cualquier entorno compatible.

TODO

Crear .dockerignore y pasar .env en tiempo de ejecución.

Hint

Nunca copies .env con secretos dentro de la imagen.

Check

docker build y docker run deben exponer /health.

Paso 9 · CI/CD: pipeline mínimo de seguridad

Antes de desplegar, el pipeline debe ejecutar tests y evaluación de regresión.

ci.yml

```yaml
name: ci

on: [push]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.11"
      - run: pip install -r requirements.txt
      - run: pytest
      - run: python run_eval.py  # optional gate
```

Objetivo

Automatizar comprobaciones antes de construir o desplegar imagen.

TODO

Añadir umbrales de evaluación y bloquear si fallan.

Hint

El pipeline puede usar stubs si no quieres gastar tokens en cada push.

Check

Un cambio que rompe business_rules debe fallar antes de producción.

Ejecución recomendada de la práctica

Primero local, después tests, evaluación y finalmente Docker.

runbook

```
# 1. Run API locally
uvicorn app:app --reload

# 2. Smoke test
curl http://127.0.0.1:8000/health

# 3. Run deterministic tests
pytest

# 4. Run offline evaluation
python run_eval.py

# 5. Build container
docker build -t ai-triage-api .

# 6. Run container
docker run --env-file .env -p 8000:8000 ai-triage-api
```

Orden didáctico

No empezamos por Docker. Primero comprobamos que la API y la lógica funcionan localmente.

Orden productivo

Tests y evaluación deberían ejecutarse antes de construir o promover una imagen.

Check final

/health, /docs, pytest y run_eval.py deben estar verdes.

Output esperado: reporte de evaluación

La práctica termina generando una tabla simple para decidir si una versión es aceptable.

eval_report.csv

```
case_id,operation_match,process_type_match,missing_fields_match,passed
EVAL-001,true,true,true,true
EVAL-002,true,true,true,true
EVAL-003,false,true,false,false

Summary:
operation_accuracy = 0.87
invalid_output_rate = 0.00
release_gate = FAILED
```

Para depurar

Qué casos fallaron y en qué criterio.

Para release

Si no supera umbral, no se despliega.

Para mejorar

Cada fallo real puede convertirse en nuevo caso de evaluación.

La evaluación convierte opiniones subjetivas en decisiones de release defendibles.

Debrief: de API funcional a servicio gobernado

La última capa del tema es hacer que el sistema sea medible, evaluable y desplegable.

1 Operabilidad

Una API LLM necesita logs, métricas y trazas útiles.

2 Evaluación

No probamos texto: probamos estados, operaciones y seguridad.

3 Versionado

Prompt, schema, modelo, reglas y config afectan a decisiones.

4 Coste

Tokens y modelo son decisiones de arquitectura.

5 Release controlado

Canary, blue-green y gates reducen riesgo.

6 Producción

Docker/CI/CD hacen el servicio reproducible y comprobable.

Una app LLM robusta no solo responde: demuestra que sigue respondiendo bien cuando el sistema cambia.

Mini-quiz de cierre

Cinco preguntas para comprobar el marco mental de LLMOps.

1. ¿Qué diferencia hay entre probar manualmente y evaluar con regression cases?

2. ¿Qué elementos versionarías además del código?

3. ¿Qué métrica mirarías si sube el coste por caso?

4. ¿Cuándo bloquearías un despliegue aunque la API arranque correctamente?

5. ¿Por qué un error de salida estructurada no debe actualizar estado?

Cierre del Tema 4

Hemos construido el camino completo: cliente, salida, estado, API y operación.

Clase 1	Clase 2	Clase 3	Clase 4	**Clase 5**
Cliente LLM medible	Salida estructurada	Contexto y estado	API propia	**LLMOps**

```
POST /cases/{case_id}/events
→ update state
→ return operation
→ trace metrics
→ evaluate baseline
→ deploy controlled release
```

end-to-end

Meta final

Una API LLM no se considera lista hasta que puede evaluarse, observarse y operarse con seguridad.

Siguiente paso profesional

Conectar métricas reales a dashboards, persistir trazas y automatizar gates de release.